

Détection de fraude à la carte bancaire

Plateforme MLOps de bout en bout — règles métier + Machine Learning

Document technique informatif

Version 1.0 · 28 June 2026

Auteur : équipe MLOps

Sommaire

Sommaire	2
1. Contexte et objectif	4
Pourquoi une approche hybride règles + ML ?	4
2. Architecture	5
Vue d'ensemble du flux temps réel	5
Principe d'unicité du feature engineering	5
Composants et responsabilités	5
3. Stack technique	6
4. Étapes du projet	7
5. Pipeline complet du projet	9
Pipeline 1 — Entraînement (offline)	9
Pipeline 2 — Inférence (online)	9
Pipeline 3 — CI / CD (automatique)	10
6. Détail du modèle ML	11
Algorithme et hyperparamètres	11
Features utilisées	11
Métriques d'évaluation	11
Résultats obtenus	12
7. Règles métier	13
8. API et exemples d'appel	14
Endpoints exposés	14
Exemple de requête (curl)	14
Réponse pour un cas frauduleux	15
9. Observabilité et exploitation	16
Logs structurés	16
Métriques Prometheus	16
Détection de drift (PSI)	16
10. Déploiement et exploitation	17
Image Docker	17
Lancement local	17
Scalabilité en production	17

11. Résultats vérifiés	18
Scénarios de smoke test	18
12. Conclusion et perspectives	19
Pistes d'évolution	19

1. Contexte et objectif

La fraude à la carte bancaire représente plusieurs milliards d'euros de pertes annuelles pour le secteur financier. Les attaques évoluent constamment (skimming, prises de contrôle de comptes, tests de cartes à grande échelle...) et exigent une réponse temps réel : chaque transaction doit être autorisée ou refusée en moins de 100 ms.

Ce projet livre une plateforme de scoring **de bout en bout**, capable de prendre une transaction en entrée et de retourner une décision parmi trois actions :

- **APPROVE** — laisser passer la transaction.
- **REVIEW** — envoyer la transaction en revue manuelle.
- **DECLINE** — refuser la transaction.

La décision repose sur la combinaison de **deux signaux complémentaires** : un moteur de règles métier (auditable, détenu par l'équipe Risque) et un modèle de Machine Learning calibré (adaptatif, détenu par l'équipe Data Science).

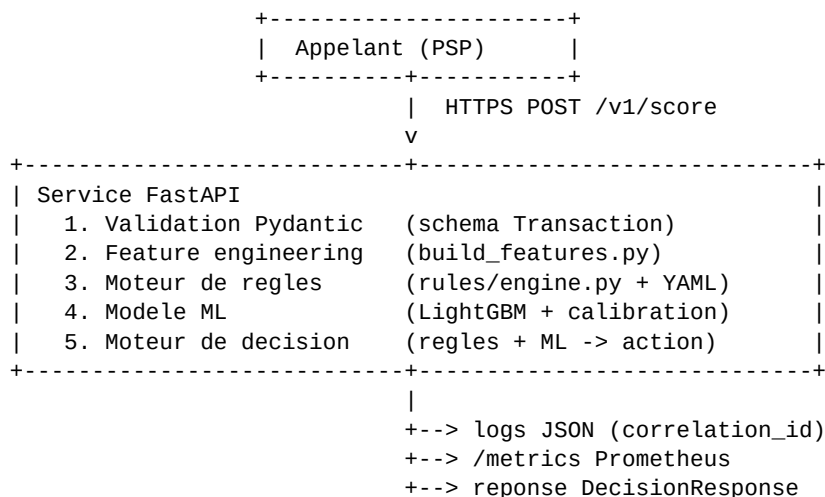
Pourquoi une approche hybride règles + ML ?

Critère	Règles seules	ML seul	Hybride (ce projet)
Couverture des fraudes connues	Forte par définition	Dépend du jeu d'entraînement	Forte (règles) + adaptative (ML)
Adaptation aux nouveaux schémas	Lente (écriture manuelle)	Rapide (réentraînement)	Rapide
Explicabilité	Excellente	Limitée	Codes de règles + probabilité ML
Contraintes réglementaires	Faciles à encoder	Difficiles à garantir	Encodées en règles HARD_BLOCK
Maîtrise du taux de faux positifs	Grossière	Réglable par seuil	Combinée (uplift + seuil)

2. Architecture

L'architecture est volontairement **simple et stateless**. Toute la complexité métier est dans les modules : l'API ne fait qu'orchestrer un appel séquentiel à cinq composants.

Vue d'ensemble du flux temps réel



Principe d'unicité du feature engineering

Le bug le plus fréquent en ML production est le **train/serve skew** : les features sont calculées différemment en mode batch (entraînement) et en mode online (service). Pour l'éliminer, *une seule fonction* `build_features(df)` est utilisée par les deux chemins. Toute évolution de feature est automatiquement disponible partout, et les tests d'intégration garantissent l'invariant.

Composants et responsabilités

Composant	Rôle	Chemin
Schémas Pydantic	Contrat strict d'entrée et de sortie	data/schemas.py
Générateur de données	5 patterns de fraude réalistes, déterministe	data/synthetic.py
Feature engineering	~20 features, source unique pour train et serve	features/build_features.py
Moteur de règles	Règles YAML déclaratives, 3 sévérités	rules/engine.py
Modèle ML	LightGBM + calibration isotonique	models/train.py, models/predict.py
Moteur de décision	Fusion règles + ML → APPROVE / REVIEW / DECLINE	decision/engine.py
API	FastAPI + lifespan + correlation_id	api/main.py
Observabilité	Métriques Prometheus + drift PSI	monitoring/

3. Stack technique

Le projet est **100 % Python 3.11**. Chaque dépendance a été choisie pour son adéquation au cas d'usage et sa maturité en production.

Couche	Outil	Justification
Langage	Python 3.11	Écosystème ML + FastAPI
Modèle ML	LightGBM 4.3	Performant sur le tabulaire déséquilibré
Calibration	scikit-learn 1.9	CalibratedClassifierCV + FrozenEstimator
Tracking ML	MLflow (SQLite)	Suivi d'expériences + artifacts
Validation	Pydantic v2	Strict, rapide, à la frontière API
API	FastAPI + uvicorn	Async, OpenAPI auto, lifespan hooks
Logs	structlog	JSON + correlation_id par contextvars
Métriques	prometheus-client	Format d'exposition standard
Config	pydantic-settings + YAML	12-factor + règles versionnées
CLI	Typer + Rich	Génération data, entraînement, scoring
Tests	pytest + httpx	Unitaire + intégration avec modèle réel
Qualité	ruff + black + mypy	Lint, format, typage
Container	Docker multi-stage	Image slim, non-root, healthcheck
CI/CD	GitHub Actions	CI sur PR, image GHCR sur tag
Notebook	Jupyter + seaborn	Analyse exploratoire des données

4. Étapes du projet

Le projet a été construit en **13 étapes** dans un ordre logique : du contrat de données au déploiement, en passant par le modèle, l'API et la documentation.

Étape 1 — Bootstrap du projet

Mise en place de l'environnement reproductible : pyproject.toml (PEP 621), Makefile pour les opérations courantes, .pre-commit-config.yaml, .gitignore et arborescence src/ standard.

Pourquoi : permettre à un nouveau développeur de cloner, installer et tester en moins de 3 minutes.

Étape 2 — Contrat d'entrée Pydantic

Définition du schéma Transaction avec contraintes au niveau des champs : amount > 0, codes ISO pour les pays, longueur exacte du MCC (Merchant Category Code), enum pour le canal.

Pourquoi : rejeter les payloads invalides à la porte d'entrée, pas en plein milieu du pipeline.

Étape 3 — Générateur de données synthétiques

Implémentation de cinq patterns de fraude : card_testing, geo_anomaly, cashout, account_takeover, high_risk_mcc. Chaque pattern reflète une attaque observée en production. Générateur déterministe (même seed = même fichier).

Pourquoi : le projet est reproductible sans donnée propriétaire, et les patterns donnent au modèle et aux règles quelque chose de pertinent à apprendre / détecter.

Étape 4 — Feature engineering (source unique)

Fonction build_features(df) qui dérive ~20 features : ratio au panier moyen 30j, z-score, vitesse 1h/24h, indicateurs nuit / week-end, geo mismatch, indicateurs CNP / ECOM / ATM, MCC à risque.

Pourquoi : utilisée à la fois en entraînement (DataFrame batch) et en serving (DataFrame à une ligne). Élimine définitivement le train/serve skew.

Étape 5 — Moteur de règles déclaratif

Fichier rules.yaml chargé au démarrage. Trois sévérités : HARD_BLOCK (force le refus), SUSPECT (ajoute une pondération à la probabilité ML), INFO (audit). 9 règles initiales : montant élevé, geo mismatch, MCC à risque, vitesse, ATM nocturne, etc.

Pourquoi : l'équipe Risque peut modifier les règles sans toucher au code. Le fichier est auditable et versionné dans Git.

Étape 6 — Pipeline d'entraînement

LightGBM avec is_unbalance=true, early stopping sur ensemble de validation temporel, puis calibration isotonique sur la même validation. MLflow log les paramètres, métriques et artifact (model.joblib + schema + stats).

Pourquoi : la calibration rend les probabilités directement interprétables comme seuils, ce qui est indispensable pour que les seuils REVIEW/DECLINE aient un sens.

Étape 7 — Moteur de décision

Logique : si une règle `HARD_BLOCK` déclenche, `DECLINE` immédiatement (court-circuit du ML, gain de latence). Sinon, chaque règle `SUSPECT` ajoute +0.05 à la probabilité ML, plafonné à +0.20. Seuils : < 0.30 = `APPROVE`, $0.30-0.70$ = `REVIEW`, ≥ 0.70 = `DECLINE`.

Pourquoi : aucune des deux composantes (règles, ML) ne peut surpasser l'autre silencieusement. La décision est traçable.

Étape 8 — Service FastAPI

Quatre endpoints : `POST /v1/score`, `POST /v1/score/batch`, `GET /healthz`, `GET /metrics`. Middleware `correlation_id` qui propage un identifiant de trace via `contextvars` dans tous les logs. Lifespan hook qui charge le modèle une seule fois au démarrage.

Pourquoi : production-grade — pas de surprise (modèle non chargé, traçabilité absente, etc.).

Étape 9 — Observabilité

Métriques Prometheus exposées sur `/metrics` : nombre de décisions par action, déclenchements par règle, distribution de la probabilité, latence (histogramme), état de santé. Utilitaire PSI (Population Stability Index) pour la détection de drift offline.

Pourquoi : un modèle qu'on ne mesure pas est un modèle qu'on ne maîtrise pas.

Étape 10 — Tests

22 tests unitaires (features, règles, décision, drift, synthétique) + 6 tests d'intégration via `TestClient` FastAPI qui entraînent un vrai modèle puis exercent l'API. Couverture totale : 88 %.

Pourquoi : les tests d'intégration attrapent les bugs que les tests unitaires manquent (chargement de modèle, train/serve skew).

Étape 11 — Containerisation

Dockerfile multi-stage : un stage builder qui installe les dépendances, un stage runtime slim qui copie le `venv` et la source, sans `build-essential`. Utilisateur non-root, healthcheck automatique, `libgomp1` pour `LightGBM`.

Pourquoi : surface d'attaque minimale, image légère, conforme aux bonnes pratiques sécurité.

Étape 12 — CI/CD GitHub Actions

Trois workflows : `ci.yml` (lint + tests + build sur chaque PR), `cd.yml` (build et push de l'image vers GHCR sur tag `v*.*.*`), `train.yml` (réentraînement manuel via `workflow_dispatch`).

Pourquoi : chaque PR est vérifiable, chaque release est reproductible, le réentraînement est un clic.

Étape 13 — Documentation

README de 12 sections, `docs/architecture.md` (composants et flux), `docs/model_card.md` (carte modèle Google-style avec limitations et fairness), `docs/runbook.md` (procédures on-call), `docs/api.md` (contrat API), notebook EDA.

Pourquoi : un projet sans documentation est un projet inutilisable au-delà de son auteur.

5. Pipeline complet du projet

Le projet comporte **deux pipelines distincts** qui partagent le même code de feature engineering :

Pipeline 1 — Entraînement (offline)

```
[1] generate-data --> data/processed/transactions_train.parquet
                        data/processed/transactions_test.parquet

[2] train-pipeline
    +-- load parquet
    +-- build_features(df)                (même fonction qu'en serving)
    +-- split temporel train / val
    +-- LightGBM.fit (early stopping)
    +-- CalibratedClassifierCV (isotonic, FrozenEstimator)
    +-- evaluation sur test (ROC AUC, PR AUC, Recall@1%FPR, top-1%)
    +-- joblib.dump --> artifacts/model.joblib
    +-- MLflow log_params + log_metrics + log_artifact
```

Pipeline 2 — Inférence (online)

```
HTTP POST /v1/score
  |
  v
[Pydantic Transaction] ----> 422 si payload invalide
  |
  v
[transaction_to_feature_row] (build_features() sur une ligne)
  |
  v
[RulesEngine.evaluate] ----> hits[]
  |
  +-- si HARD_BLOCK ----> on saute le ML, action = DECLINE
  |
  v
[FraudModel.predict_one] ----> p_ml
  |
  v
[DecisionEngine.decide] ----> action, risk_score, reasons
  |
  v
[DecisionResponse JSON] ----> reponse au client
```

Pipeline 3 — CI / CD (automatique)

Push / Pull Request sur main

|

v

[ci.yml]

```
+-- ruff check (lint)
+-- black --check (format)
+-- mypy (typage, informatif)
+-- pytest --cov (28 tests, 88% couverture)
+-- docker build (sans push)
```

Tag v*.*.*

|

v

[cd.yml]

```
+-- build multi-stage
+-- push image vers ghcr.io/owner/repo:vX.Y.Z
```

Action manuelle (workflow_dispatch)

|

v

[train.yml]

```
+-- generate-data
+-- train
+-- upload artifacts (model.joblib, mlruns)
```

6. Détail du modèle ML

Algorithme et hyperparamètres

Le modèle est un **LightGBM binaire** (gradient boosting sur arbres) avec gestion native du déséquilibre de classes. Hyperparamètres versionnés dans configs/model.yaml :

```
lightgbm:
  objective: binary
  metric: auc
  learning_rate: 0.05
  num_leaves: 64
  min_data_in_leaf: 50
  feature_fraction: 0.85
  bagging_fraction: 0.85
  bagging_freq: 5
  lambda_l2: 1.0
  is_unbalance: true

training:
  num_boost_round: 600
  early_stopping_rounds: 40
  val_fraction: 0.15
  calibration_method: isotonic
```

Features utilisées

21 features groupées en cinq familles :

- **Montant** : amount, amount_log, amount_ratio_avg_30d, amount_zscore_30d
- **Vélocité** : n_tx_last_1h, n_tx_last_24h, velocity_score, is_burst_1h
- **Temporel** : hour, is_night (0-5h), is_weekend
- **Géographique** : is_geo_mismatch (carte vs commerçant)
- **Canal / risque** : is_cnp, is_ecom, is_atm, is_high_risk_mcc
- **Client** : customer_age_days, card_age_days, amount_avg_30d, amount_std_30d, distinct_countries_last_24h

Métriques d'évaluation

Métrique	Pourquoi on la suit
ROC AUC	Qualité globale du ranking (référence générale).
PR AUC	Métrique principale — robuste au déséquilibre.
Recall @ 1% FPR	Combien de fraudes on attrape en gardant un taux de fausse alarme faible.
Precision @ top 1%	Charge de revue manuelle : sur le 1% le plus risqué, quelle proportion est réellement frauduleuse.
F1 @ 0.5	Référence seulement ; les vrais seuils de production viennent de model.yaml.

Résultats obtenus

Métrique	Valeur
ROC AUC	0,991
PR AUC	0,983
Recall @ 1% FPR	0,98
Precision @ top 1%	1,00
F1 @ seuil par défaut	0,991
Latence p99 /v1/score	< 20 ms (objectif : < 100 ms)
Couverture des tests	88 %

7. Règles métier

Les règles sont déclarées dans configs/rules.yaml. Trois sévérités :

- **HARD_BLOCK** — force DECLINE quelle que soit la probabilité ML.
- **SUSPECT** — pousse la probabilité ML vers le haut, peut router vers REVIEW.
- **INFO** — audit uniquement, sans impact sur la décision.

Code	Sévérité	Déclencheur
R001_HIGH_AMOUNT_CNP	SUSPECT	Card-Not-Present $\geq$ 1000 €
R002_VERY_HIGH_AMOUNT	HARD_BLOCK	Transaction $\geq$ 5000 €
R003_GEO_MISMATCH_ECOM	SUSPECT	Pays carte $\neq$ pays commerçant en ECOM
R004_VELOCITY_BURST	HARD_BLOCK	$\geq$ 8 transactions dans l'heure
R005_HIGH_RISK_MCC	SUSPECT	MCC à risque (jeu d'argent, adulte)
R006_NIGHT_ATM	SUSPECT	Retrait DAB entre minuit et 5 h
R007_NEW_CARD_LARGE_AMOUNT	SUSPECT	Carte < 7 jours et montant $\geq$ 300 €
R008_MULTI_COUNTRY_24H	HARD_BLOCK	$\geq$ 3 pays distincts en 24 h
R009_ZSCORE_OUTLIER	SUSPECT	Z-score du montant $\geq$ 6

8. API et exemples d'appel

Endpoints exposés

Méthode	Chemin	Description
POST	/v1/score	Scorer une transaction
POST	/v1/score/batch	Scorer jusqu'à 100 transactions
GET	/healthz	Liveness + readiness (modèle chargé ?)
GET	/metrics	Format d'exposition Prometheus
GET	/docs	Swagger UI (OpenAPI)

Exemple de requête (curl)

```
curl -X POST http://localhost:8000/v1/score \
-H "Content-Type: application/json" \
-H "X-Correlation-ID: demo-001" \
-d '{
  "transaction_id": "TX_DEMO_001",
  "timestamp": "2025-06-15T03:10:00Z",
  "customer_id": "C00000001",
  "card_id": "K00000001",
  "amount": 1500.0,
  "currency": "EUR",
  "merchant_id": "M9999999",
  "merchant_country": "NG",
  "merchant_mcc": "7995",
  "card_country": "FR",
  "channel": "ECOM",
  "is_cnp": true,
  "customer_age_days": 900,
  "card_age_days": 400,
  "n_tx_last_1h": 4,
  "n_tx_last_24h": 10,
  "amount_avg_30d": 70.0,
  "amount_std_30d": 20.0,
  "distinct_countries_last_24h": 2
}'
```

Réponse pour un cas frauduleux

```
{
  "transaction_id": "TX_DEMO_001",
  "action": "DECLINE",
  "fraud_probability": 1.0,
  "risk_score": 1000,
  "reasons": [
    "rule:R001_HIGH_AMOUNT_CNP",
    "rule:R003_GEO_MISMATCH_ECOM",
    "rule:R005_HIGH_RISK_MCC",
    "ml:p=1.000>=decline_threshold"
  ],
  "rule_hits": [...],
  "model_version": "20260628-181316",
  "latency_ms": 13.09
}
```

9. Observabilité et exploitation

Logs structurés

Tous les logs sont émis en **JSON** (structlog) et portent un `correlation_id` propagé par `contextvar` dans tous les composants (feature engineering, règles, ML, décision). Un appel HTTP unique peut donc être tracé de bout en bout dans Loki, Elasticsearch ou CloudWatch.

Métriques Prometheus

Métrique	Type	Utilité
<code>fraud_decisions_total{action}</code>	Counter	KPI produit : ratio APPROVE/REVIEW/DECLINE
<code>fraud_rule_hits_total{code,severity}</code>	Counter	Suivi des règles : détection d'une règle muette ou folle
<code>fraud_probability</code>	Histogramme	Distribution des probabilités, signal de drift
<code>fraud_inference_latency_seconds</code>	Histogramme	SLO de latence (p99 < 100 ms)
<code>fraud_service_healthy</code>	Gauge	État de santé : modèle chargé ?

Détection de drift (PSI)

Le Population Stability Index (PSI) est le standard du secteur financier pour mesurer un décalage de distribution entre une référence (training) et un échantillon courant (production). Seuils :

- **PSI < 0,10** — distribution stable.
- **0,10 ≤ PSI < 0,25** — décalage modéré, à investiguer.
- **PSI ≥ 0,25** — décalage majeur, réentraînement requis.

10. Déploiement et exploitation

Image Docker

Le Dockerfile est **multi-stage** (builder + runtime) et utilise un utilisateur non-root. Seule libgomp1 est conservée dans l'image finale (dépendance native de LightGBM).

Lancement local

```
# Installation
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

# Pipeline complet
make data      # genere les transactions synthetiques
make train     # entraine le modele
make serve     # lance l'API sur :8000

# Ou via Docker
make docker-build
make docker-up  # API sur :8000 avec artifacts montes en read-only
```

Scalabilité en production

Préoccupation	Approche
Débit	Horizontale — API stateless, ajouter des répliques derrière un load balancer ou un HPA k8s.
Latence	LightGBM monoligne < 5 ms. Le coût dominant est Pydantic + feature build.
Chargement du modèle	Une seule fois au démarrage (lifespan). Aucun I/O par requête.
Features de vélocité	Calculées en amont par un feature store (Redis, Feast). L'API les reçoit dans le payload.
Réentraînement	Airflow, Kubeflow ou workflow_dispatch GitHub Actions.
Déploiement progressif	Shadow scoring : nouveau modèle en parallèle de l'ancien, on log les deux et on compare.

11. Résultats vérifiés

Le projet a été exécuté de bout en bout dans un environnement propre avec les résultats suivants :

Vérification	Résultat
Lint (ruff)	Clean
Format (black --check)	Clean
Tests unitaires + intégration	28 / 28 verts, couverture 88 %
Modèle — ROC AUC	0,991
Modèle — PR AUC	0,983
Modèle — Recall @ 1% FPR	0,98
Smoke test 4 scénarios métier	Tous corrects
Latence end-to-end (smoke)	9 – 17 ms par requête

Scénarios de smoke test

Scénario	Action attendue	Résultat
POS local épicerie	APPROVE	APPROVE (p ≈ 0)
ECOM + geo mismatch + MCC risque + nuit	DECLINE	DECLINE (4 règles SUSPECT + ML)
Montant 7500 € (> 5000)	DECLINE	DECLINE (R002 HARD_BLOCK)
12 transactions en 1 heure	DECLINE	DECLINE (R004 HARD_BLOCK)

12. Conclusion et perspectives

Le projet livre une plateforme MLOps **opérationnelle**, auditable et industrialisée. Les choix d'architecture (hybride règles + ML, feature engineering unifié, calibration des probabilités, observabilité dès le jour 1) sont alignés avec les pratiques observées dans les fintech matures.

Pistes d'évolution

- **Modèle challenger** (XGBoost ou réseau peu profond) avec shadow scoring intégré à l'API.
- **Optimisation automatique des seuils** sur courbe coût/bénéfice métier.
- **SHAP par transaction** exposé dans /v1/score pour l'équipe Fraud.
- **Streaming Kafka** en complément du mode REST.
- **Feature store Feast** pour les features de vélocité (au lieu du payload).
- **Manifest Kubernetes** + Helm chart (HPA, PDB, ServiceMonitor).
- **Versioning Data + Modèle** avec DVC ou LakeFS, artifact store S3.
- **Authentification mTLS ou JWT** sur /v1/score.
- **Dashboard Grafana** prêt à l'emploi pour les métriques Prometheus.
- **Chaos tests** (modèle absent, config corrompue, latence dégradée) pour valider le runbook.

Document de référence — équipe MLOps